

#	Description	Level
16.2.4	Verify that logs can be read and correlated by the log processor that is in use, preferably by using a common logging format.	2
16.2.5	Verify that when logging sensitive data, the application enforces logging based on the data's protection level. For example, it may not be allowed to log certain data, such as credentials or payment details. Other data, such as session tokens, may only be logged by being hashed or masked, either in full or partially.	2

V16.3 Security Events

This section defines requirements for logging security-relevant events within the application. Capturing these events is critical for detecting suspicious behavior, supporting investigations, and fulfilling compliance obligations.

This section outlines the types of events that should be logged but does not attempt to provide exhaustive detail. Each application has unique risk factors and operational context.

Note that while ASVS includes logging of security events in scope, alerting and correlation (e.g., SIEM rules or monitoring infrastructure) are considered out of scope and are handled by operational and monitoring systems.

#	Description	Level
16.3.1	Verify that all authentication operations are logged, including successful and unsuccessful attempts. Additional metadata, such as the type of authentication or factors used, should also be collected.	2
16.3.2	Verify that failed authorization attempts are logged. For L3, this must include logging all authorization decisions, including logging when sensitive data is accessed (without logging the sensitive data itself).	2
16.3.3	Verify that the application logs the security events that are defined in the documentation and also logs attempts to bypass the security controls, such as input validation, business logic, and anti-automation.	2
16.3.4	Verify that the application logs unexpected errors and security control failures such as backend TLS failures.	2

V16.4 Log Protection

Logs are valuable forensic artifacts and must be protected. If logs can be easily modified or deleted, they lose their integrity and become unreliable for incident investigations or legal proceedings. Logs may expose internal application behavior or sensitive metadata, making them an attractive target for attackers.

This section defines requirements to ensure that logs are protected from unauthorized access, tampering, and disclosure, and that they are safely transmitted and stored in secure, isolated systems.

#	Description	Level
16.4.1	Verify that all logging components appropriately encode data to prevent log injection.	2
16.4.2	Verify that logs are protected from unauthorized access and cannot be modified.	2
16.4.3	Verify that logs are securely transmitted to a logically separate system for analysis, detection, alerting, and escalation. The aim is to ensure that if the application is breached, the logs are not compromised.	2

V16.5 Error Handling

This section defines requirements to ensure that applications fail gracefully and securely without disclosing sensitive internal details.

#	Description	Level
16.5.1	Verify that a generic message is returned to the consumer when an unexpected or security-sensitive error occurs, ensuring no exposure of sensitive internal system data such as stack traces, queries, secret keys, and tokens.	2
16.5.2	Verify that the application continues to operate securely when external resource access fails, for example, by using patterns such as circuit breakers or graceful degradation.	2
16.5.3	Verify that the application fails gracefully and securely, including when an exception occurs, preventing fail-open conditions such as processing a transaction despite errors resulting from validation logic.	2

#	Description	Level
16.5.4	Verify that a “last resort” error handler is defined which will catch all unhandled exceptions. This is both to avoid losing error details that must go to log files and to ensure that an error does not take down the entire application process, leading to a loss of availability.	3

Note: Certain languages, (including Swift, Go, and through common design practice, many functional languages,) do not support exceptions or last-resort event handlers. In this case, architects and developers should use a pattern, language, or framework-friendly way to ensure that applications can securely handle exceptional, unexpected, or security-related events.

References

For more information, see also:

- OWASP Web Security Testing Guide: Testing for Error Handling
- OWASP Authentication Cheat Sheet section about error messages
- OWASP Logging Cheat Sheet
- OWASP Application Logging Vocabulary Cheat Sheet

V17 WebRTC

Control Objective

Web Real-Time Communication (WebRTC) enables real-time voice, video, and data exchange in modern applications. As adoption increases, securing WebRTC infrastructure becomes critical. This section provides security requirements for stakeholders who develop, host, or integrate WebRTC systems.

The WebRTC market can be broadly categorized into three segments:

1. **Product Developers:** Proprietary and open-source vendors that create and supply WebRTC products and solutions. Their focus is on developing robust and secure WebRTC technologies that can be used by others.
2. **Communication Platforms as a Service (CPaaS):** Providers that offer APIs, SDKs, and the necessary infrastructure or platforms to enable WebRTC functionalities. CPaaS providers may use products from the first category or develop their own WebRTC software to offer these services.
3. **Service Providers:** Organizations that leverage products from product developers or CPaaS providers, or develop their own WebRTC solutions. They create and implement applications

for online conferencing, healthcare, e-learning, and other domains where real-time communication is crucial.

The security requirements outlined here are primarily focused on Product Developers, CPaaS, and Service Providers who:

- Utilize open-source solutions to build their WebRTC applications.
- Use commercial WebRTC products as part of their infrastructure.
- Use internally developed WebRTC solutions or integrate various components into a cohesive service offering.

It is important to note that these security requirements do not apply to developers who exclusively use SDKs and APIs provided by CPaaS vendors. For such developers, the CPaaS providers are typically responsible for most of the underlying security concerns within their platforms, and a generic security standard like ASVS may not fully address their needs.

V17.1 TURN Server

This section defines security requirements for systems that operate their own TURN (Traversal Using Relays around NAT) servers. TURN servers assist in relaying media in restrictive network environments but can pose risks if misconfigured. These controls focus on secure address filtering and protection against resource exhaustion.

#	Description	Level
17.1.1	Verify that the Traversal Using Relays around NAT (TURN) service only allows access to IP addresses that are not reserved for special purposes (e.g., internal networks, broadcast, loopback). Note that this applies to both IPv4 and IPv6 addresses.	2
17.1.2	Verify that the Traversal Using Relays around NAT (TURN) service is not susceptible to resource exhaustion when legitimate users attempt to open a large number of ports on the TURN server.	3

V17.2 Media

These requirements only apply to systems that host their own WebRTC media servers, such as Selective Forwarding Units (SFUs), Multipoint Control Units (MCUs), recording servers, or gateway servers. Media servers handle and distribute media streams, making their security critical to protect communication between peers. Safeguarding media streams is paramount in WebRTC applications to prevent eavesdropping, tampering, and denial-of-service attacks that could compromise user privacy and communication quality.

In particular, it is necessary to implement protections against flood attacks such as rate limiting, validating timestamps, using synchronized clocks to match real-time intervals, and managing buffers to prevent overflow and maintain proper timing. If packets for a particular media session arrive too quickly, excess packets should be dropped. It is also important to protect the system from malformed packets by implementing input validation, safely handling integer overflows, preventing buffer overflows, and employing other robust error-handling techniques.

Systems that rely solely on peer-to-peer media communication between web browsers, without the involvement of intermediate media servers, are excluded from these specific media-related security requirements.

This section refers to the use of Datagram Transport Layer Security (DTLS) in the context of WebRTC. A requirement related to having a documented policy for the management of cryptographic keys can be found in the “Cryptography” chapter. Information on approved cryptographic methods can be found either in the Cryptography Appendix of the ASVS or in documents such as NIST SP 800-52 Rev. 2 or BSI TR-02102-2 (Version 2025-01).

#	Description	Level
17.2.1	Verify that the key for the Datagram Transport Layer Security (DTLS) certificate is managed and protected based on the documented policy for management of cryptographic keys.	2
17.2.2	Verify that the media server is configured to use and support approved Datagram Transport Layer Security (DTLS) cipher suites and a secure protection profile for the DTLS Extension for establishing keys for the Secure Real-time Transport Protocol (DTLS-SRTP).	2
17.2.3	Verify that Secure Real-time Transport Protocol (SRTP) authentication is checked at the media server to prevent Real-time Transport Protocol (RTP) injection attacks from leading to either a Denial of Service condition or audio or video media insertion into media streams.	2
17.2.4	Verify that the media server is able to continue processing incoming media traffic when encountering malformed Secure Real-time Transport Protocol (SRTP) packets.	2
17.2.5	Verify that the media server is able to continue processing incoming media traffic during a flood of Secure Real-time Transport Protocol (SRTP) packets from legitimate users.	3
17.2.6	Verify that the media server is not susceptible to the “ClientHello” Race Condition vulnerability in Datagram Transport Layer Security (DTLS) by checking if the media server is publicly known to be vulnerable or by performing the race condition test.	3

#	Description	Level
17.2.7	Verify that any audio or video recording mechanisms associated with the media server are able to continue processing incoming media traffic during a flood of Secure Real-time Transport Protocol (SRTP) packets from legitimate users.	3
17.2.8	Verify that the Datagram Transport Layer Security (DTLS) certificate is checked against the Session Description Protocol (SDP) fingerprint attribute, terminating the media stream if the check fails, to ensure the authenticity of the media stream.	3

V17.3 Signaling

This section defines requirements for systems that operate their own WebRTC signaling servers. Signaling coordinates peer-to-peer communication and must be resilient against attacks that could disrupt session establishment or control.

To ensure secure signaling, systems must handle malformed inputs gracefully and remain available under load.

#	Description	Level
17.3.1	Verify that the signaling server is able to continue processing legitimate incoming signaling messages during a flood attack. This should be achieved by implementing rate limiting at the signaling level.	2
17.3.2	Verify that the signaling server is able to continue processing legitimate signaling messages when encountering malformed signaling message that could cause a denial of service condition. This could include implementing input validation, safely handling integer overflows, preventing buffer overflows, and employing other robust error-handling techniques.	2

References

For more information, see also:

- The WebRTC DTLS ClientHello DoS is best documented at Enable Security's blog post aimed at security professionals and the associated white paper aimed at WebRTC developers
- RFC 3550 - RTP: A Transport Protocol for Real-Time Applications
- RFC 3711 - The Secure Real-time Transport Protocol (SRTP)
- RFC 5764 - Datagram Transport Layer Security (DTLS) Extension to Establish Keys for the Secure Real-time Transport Protocol (SRTP))

- RFC 8825 - Overview: Real-Time Protocols for Browser-Based Applications
- RFC 8826 - Security Considerations for WebRTC
- RFC 8827 - WebRTC Security Architecture
- DTLS-SRTP Protection Profiles

Appendix A: Glossary

- **Absolute Maximum Session Lifetime** –Also referred to as “Overall Timeout” by NIST, this is the maximal amount of time a session can remain active following authentication regardless of user interaction. This is a component of session expiration.
- **Allowlist** –A list of permitted data or operations, for example, a list of characters that are allowed to perform input validation.
- **Anti-forgery token** –A mechanism by which one or more tokens are passed in a request and validated by the application server to ensure that the request has come from an expected endpoint.
- **Application Security** –Application-level security focuses on the analysis of components that comprise the application layer of the Open Systems Interconnection Reference Model (OSI Model), rather than focusing on for example the underlying operating system or connected networks.
- **Application Security Verification** –The technical assessment of an application against the OWASP ASVS.
- **Application Security Verification Report** –A report that documents the overall results and supporting analysis produced by the verifier for a particular application.
- **Authentication** –The verification of the claimed identity of an application user.
- **Automated Verification** –The use of automated tools (either dynamic analysis tools, static analysis tools, or both) that use vulnerability signatures to find problems.
- **Black box testing** –A method of software testing that examines the functionality of an application without peering into its internal structures or workings.
- **Common Weakness Enumeration (CWE)** –A community-developed list of common software security weaknesses. It serves as a common language, a measuring stick for software security tools, and a baseline for weakness identification, mitigation, and prevention efforts.
- **Component** –A self-contained unit of code, with associated disk and network interfaces that communicates with other components.
- **Credential Service Provider (CSP)** –Also called an Identity Provider (IdP). A source of user data which may be used as an authentication source by other applications.
- **Cross-Site Script Inclusion (XSSI)** –A variant of Cross-Site Scripting (XSS) attack in which a web application retrieves malicious code from an external resource and includes that code as part of its own content.
- **Cross-Site Scripting (XSS)** –A security vulnerability typically found in web applications allowing the injection of client-side scripts into content.

- **Cryptographic module** –Hardware, software, and/or firmware that implements cryptographic algorithms and/or generates cryptographic keys.
- **Cryptographically secure pseudo-random number generator** (CSPRNG) - A pseudorandom number generator with properties that make it suitable for use in cryptography, also referred to as a cryptographic random number generator (CRNG).
- **Datagram Transport Layer Security** (DTLS) –A cryptographic protocol which provides communication security over a network connection. It is based on the TLS protocol but adapted for protecting datagram-oriented protocols (usually over UDP). Defined in RFC 9147 for DTLS 1.3.
- **Datagram Transport Layer Security Extension to Establish Keys for the Secure Real-time Transport Protocol** (DTLS-SRTP) –A mechanism for using a DTLS handshake for establishing key material for a SRTP session. Defined in RFC 5764.
- **Design Verification** –The technical assessment of the security architecture of an application.
- **Dynamic Application Security Testing** (DAST) –Technologies are designed to detect conditions indicative of a security vulnerability in an application in its running state.
- **Dynamic Verification** –The use of automated tools that use vulnerability signatures to find problems during the execution of an application.
- **Fast IDentity Online** (FIDO) –A set of authentication standards that allow a variety of different authentication methods to be used including biometrics, Trusted Platform Modules (TPMs), USB security tokens, etc.
- **Hardware Security Module** (HSM) –Hardware component that stores cryptographic keys and other secrets in a protected manner.
- **Hibernate Query Language** (HQL) –A query language that is similar in appearance to SQL used by the Hibernate ORM library.
- **HTTP Strict Transport Security** (HSTS) –An policy which instructs the browser to only connect to the domain returning the header via TLS and when a valid certificate is presented. It is activated using the Strict-Transport-Security response header field.
- **HyperText Transfer Protocol** (HTTP) –An application protocol for distributed, collaborative, hypermedia information systems. It is the foundation of data communication for the World Wide Web.
- **HyperText Transfer Protocol over SSL/TLS** (HTTPS) –A method of securing HTTP communication by encrypting it using Transport Layer Security (TLS).
- **Identity Provider** (IdP) –Also called a Credential Service Provider (CSP) in NIST references. An entity that provides an authentication source for other applications.
- **Inactivity Timeout** –This is the length of time a session can remain active in the absence of user interaction with the application. This is a component of session expiration.
- **Input Validation** –The canonicalization and validation of untrusted user input.
- **JSON Web Token** (JWT) –RFC 7519 defines a standard for a JSON data object made up of a header section which explains how to validate the object, a body section containing a set of claims, and a signature section which contains a digital signature which can be used to validate the contents of the body section. It is a type of self-contained token.
- **Local File Inclusion** (LFI) - An attack that exploits vulnerable file inclusion procedures in an

application, leading to the inclusion of local files already present on the server.

- **Malicious Code** –Code introduced into an application during its development unbeknownst to the application owner, which circumvents the application’s intended security policy. Not the same as malware such as a virus or worm!
- **Malware** –Executable code that is introduced into an application during runtime without the knowledge of the application user or administrator.
- **Message authentication code (MAC)** - A cryptographic checksum on data, computed by a MAC generation algorithm, that is used to provide assurance on its integrity and authenticity.
- **Multi-factor authentication (MFA)** –Authentication which includes two or more of the single factors.
- **Mutual TLS (mTLS)** –See TLS client authentication.
- **Object-relational Mapping (ORM)** –A system used to allow a relational/table-based database to be referenced and queried within an application program using an application-compatible object model.
- **One-time Password (OTP)** –A password that is uniquely generated to be used on a single occasion.
- **Open Worldwide Application Security Project (OWASP)** –The Open Worldwide Application Security Project (OWASP) is a worldwide free and open community focused on improving the security of application software. Our mission is to make application security “visible,”so that people and organizations can make informed decisions about application security risks. See: <https://www.owasp.org/>.
- **Password-Based Key Derivation Function 2 (PBKDF2)** –A special one-way algorithm used to create a strong cryptographic key from an input text (such as a password) and an additional random salt value and can therefore be used to make it harder to crack a password offline if the resulting value is stored instead of the original password.
- **Public Key Infrastructure (PKI)** –An arrangement that binds public keys with respective identities of entities. The binding is established through a process of registration and issuance of certificates at and by a certificate authority (CA).
- **Public Switched Telephone Network (PSTN)** –The traditional telephone network that includes both fixed-line telephones and mobile telephones.
- **Real-time Transport Protocol (RTP)** and **Real-time Transport Control Protocol (RTCP)** –Two protocols used in association for transporting multimedia streams. Used by the WebRTC stack. Defined in RFC 3550.
- **Reference Token** –A type of token that acts as a pointer or identifier to state or metadata stored on a server, sometimes referred to as random tokens or opaque tokens. Unlike self-contained tokens, which embed some of their relevant data within the token itself, reference tokens contain no intrinsic information, instead relying on the server for context. The reference token will either be or contain a session identifier.
- **Relying Party (RP)** –Generally an application which is relying on a user having authenticated against a separate authentication provider. The application relies on some sort of token or set of signed assertions provided by that authentication provider to trust that the user is who they

say they are.

- **Remote File Inclusion** (RFI) - An attack that exploits vulnerable inclusion procedures in the application, resulting in the inclusion of remote files.
- **Scalable Vector Graphics** (SVG) -An XML-based markup language for describing two-dimensional based vector graphics.
- **Secure Real-time Transport Protocol** (SRTP) and **Secure Real-time Transport Control Protocol** (SRTCP) -A profile of the RTP and RTCP protocols providing support for message encryption, authentication and integrity protection. Defined in RFC 3711.
- **Security Architecture** -An abstraction of an application's design that identifies and describes where and how security controls are used, and also identifies and describes the location and sensitivity of both user and application data.
- **Security Assertion Markup Language** (SAML) -An open standard for single sign-on authentication based on passing signed assertions (usually XML objects) between the identity provider and the relying party.
- **Security Configuration** -The runtime configuration of an application that affects how security controls are used.
- **Security Control** -A function or component that performs a security check (e.g., an authorization check) or when called results in a security effect (e.g., generating an audit record).
- **Security information and event management** (SIEM) - A system for threat detection, compliance and security incident management through the collection and analysis of security-related data from various sources within an organization's IT infrastructure.
- **Self-Contained Token** -A token that encapsulates one or more attributes that do not rely on server-side state or other external storage. These tokens ensure the authenticity and integrity of their contained attributes, enabling secure, "stateless" information exchange across systems. Self-contained tokens are generally secured using cryptographic techniques, such as digital signatures or message authentication codes (MACs), to ensure the authenticity, integrity, and in some cases the confidentiality of its data. Common examples include SAML Assertions and JWTs.
- **Server-side Request Forgery** (SSRF) -An attack that abuses functionality on the server to read or update internal resources. The attacker supplies or modifies a URL, which the code running on the server will read or submit data to.
- **Session Description Protocol** (SDP) -A message format for setting up multimedia session (used for example in WebRTC). Defined in RFC 4566.
- **Session Identifier** or **Session ID** -A key which identifies a stateful session stored at the back end. Will be transferred to and from the client either as or inside a "Reference Token".
- **Session Token** -A "catch-all" phrase used in this standard to refer to the token or value used in either stateless session mechanisms (which use a self-contained token) or stateful session mechanisms (which use a reference token).
- **Session Traversal Utilities for NAT** (STUN) -A protocol used to assist NAT traversal in order to establish peer-to-peer communications. Defined in RFC 3489.
- **Single-factor authenticator** -A mechanism to check that a user is authenticated. It should ei-